ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

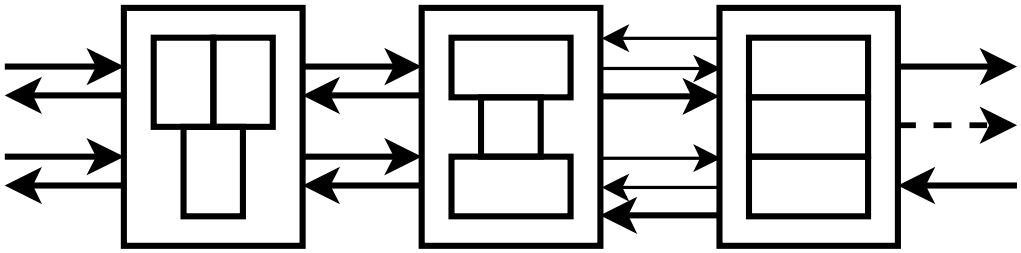
Institut für Integrierte Systeme
Integrated Systems Laboratory

Department of Information Technology and
Electrical Engineering

Spring Semester 2025

Low-Power AXI Serial Link

Semester Project



Filippo Christen
fchrist@student.ethz.ch

May 2025

Supervisors: Tim Fischer, fischeti@iis.ee.ethz.ch
Sina Arjmandpour, sarjmadpour@iis.ee.ethz.ch
Professor: Prof. Dr. L. Benini, lbenini@iis.ee.ethz.ch

Acknowledgements

I would like to express my deepest gratitude towards Tim Fischer and Sina Arjmandpour for their crucial insigths and feedback during the completion of the project. Furthermore I would like to thank Prof. Dr. Luca Benini for giving me the chance to work on such an interesting project providing me with a unique academic and personal learning experience.

This work also benefitted from the use of ChatGPT by OpenAI, which was employed for syntax clarification, bug identification and refining the phrasing of bullet points into cohesive sentences. All code, figures, problem-solving approaches and core content, however, were developed independently.

Abstract

The ever growing complexity of Internet-of-Things (IoT) systems and near-sensor processing algorithms drives a corresponding increase in off-chip data transfer demands, while stringent battery-powered operation imposes tight energy budgets. To address the competing requirements of high bandwidth and low power consumption, this work presents a low-power serial link with an integrated AXI4 interface, featuring a novel mixed-signal PHY and a custom PHY wrapper. The serial link implements credit-based flow control withouth requiring on-link handshaking. The mixed-signal PHY shows achieves up to 4 Gbps theoretical throughput at just 2 mW, corresponding to 500 fJ/bit: an $\approx 3.2\times$ improvement over a prior all-digital implementation. Power efficiency is further enhanced by an idle mode that disables the PHY when no valid data is available to be transmitted. Functional simulations confirm correct operation and expose corner case limitations, such as invalid-byte registration during brief transmission interruptions. These results demonstrate the viability of high-throughput, energy-efficient AXI-interfaced serial links for edge applications, leveraging the novel PHY.

Contents

List of Figures

Chapter 1

Introduction

IoT systems and near-sensor processing algorithms are continuously increasing in complexity. This rise in complexity also leads to a significant increase in the amount of data that needs to be moved within these systems.

Such architectures typically consist of multiple interconnected chips, off-chip memories and sensors. Consequently, they require data to be transfered off-chip, across these various devices. A high data rate across devices is therefore crucial for the efficient operation of such systems.

Furthemore, IoT systems often operate on battery power, which imposes tight constraints on power and energy consumption. Therefore, in edge applications, the energy efficiency of the components and their interconnections becomes a critical design constraint.

Achieving high bandwidth at low power consumption presents a set of competing requirements for any interconnection solution. Serial links are commonly employed as they provide a practical compromise to satisfy these constraints.

In this work, a low-power serial link incorporating an AXI-4 interface is presented.

1.1 Previous work

MBus, a bus interconnect that connects multiple modules while operating at low power is presented in [2]. The proposed design is scalable and low power, achieving particularly low standby power by allowing each connected module to have fewer than 50 gates active during standby. While operational, the implemented version of *MBus* supports data rates of up to 7.1 Mbps.

An innovative I2C-compatible asynchronous design with switcheable communication speed is demonstrated in [3]. It supports a maximum data rate of 3.4 Mbps. The design emphasizes ultra-low power consumptioin and incurs only a small hardware cost.

Another data link solution is presented in [4], which employs low data rate communication via a Die-to-Die Simplex Link (D2DSL). It achieves a maximum write data rate of 55.56 Mbps and a maximum read data rate of 34.78 Mbps at a burst length of 16.

The design described in [5] is capable of transmitting high-bandwidth data at low power and is intended for chip-to-chip communication. This is accomplished using a software-controlled low-swind transciever that connects directly to the SoC's micro-DMA engine as an additional channel. The data link targets a bandwidth of 0.8 Gbps.

Although these designs offer viable approaches for low-power data transmission, they either fall short of the high data rates demanded by modern IoT applications or lack compatibility with AXI-centric systems.

1.2 Contribution

In this work, a low-power AXI-interfaced serial link utilizing a novel mixed-signal PHY is presented. This novel PHY achieves significantly higher data rates than the previous designs introduced in [2], [3], [4] and [5] while maintaining low power consumption.

To further reduce power consumption, the serial link can place the transmitter PHY into a low power idle mode when not actively transmitting data.

1.2.1 AXI4 Interface

The Advanced eXtensible Interface (AXI) is a widely used interconnect standard for on-chip communication [1]. AXI offers high throughput and flexible data transfer capabilities.

The serial link developed in this work integrates the AXI interface directly into the serial link. This enables compatibility with SoC architectures that use AXI as their on-chip communication standard.

A similar link was already developed at the IIS lab at ETH Zürich [6], however, the all-digital PHY used in that design has proven to be a significant performance bottleneck for high-bandwidth applications.

1.2.2 Novel non-digital PHY

The high data rate of the serial link presented in this work, is achieved by means of a novel non-digital PHY capable of reaching data rates up to 4 Gbps. This improvement in data rate is made possible by the incorporation of advanced mixed-signal techniques in the PHY's implementation, which significantly enhance both throughput and power enfficiency.

Chapter 2

Theory

The low-power serial link presented in this work, directly interfaces with the AXI protocol. It translates AXI transfers into AXI-Stream, which is then serialized and transmitted over the link. An overview of the AXI and AXI-Stream protocols is provided in the following.

2.1 Advanced eXtensible Interface (AXI)

Advanced eXtensible Interface (AXI) is a protocol that supports high-performance, high frequency system designs. It is often used as an intra-chip communication standard.

The following description closely follows the AMBA AXI-4 documentation [1].

2.1.1 AXI Basics

AXI is an on-chip communication standard designed to support high-bandwidth and low-latency system designs. It is particularly suitable for high-frequency operation without requiring complex bridge components, making it well-suited for scalable and efficient SoC interconnects.

One of AXI's key strengths lies in its versatility. It meets the interface requirements of a wide range of components and provides implementation flexibility for different interconnect architectures. Additionally, it is backward-compatible with existing Advanced High-performance Bus (AHB) and Advanced Peripheral Bus (APB) interfaces, facilitating integration with legacy systems.

The protocol is especially effective for memory controllers, including those with high initial access latency, and supports efficient Direct Memory Access (DMA) thanks to its separation of read and write data channels.

AXI transactions are burst-based, where only the start address needs to be issued, improving efficiency in data transfers. It supports unaligned data transfers through the use of byte strobes, and provides separate address/control and data phases. This separation enables better pipelining an parallelism.

Moreover, AXI allows for multiple outstanding addresses and supports out-of-order transaction completion, which are critical features for high-performance applications. To help with timing closure in high-frequency designs, the protocol also permits the straightforward insertion of register stages in the data path.

2.1.2 Channels

The AXI protocol defines five independent transaction channels:

- Read address (AR)
- Read data (R)
- Write address (AW)
- Write data (W)
- Write response (B)

Each of these channels operates independently and consists of a set of information signals, along with `VALID` and `READY` signals. These signals implement a two-way handshake mechanism, ensuring reliable data transfer between the master and slave components.

AXI separates the address and data phases for both read and write transactions. The read and write operations each have their own dedicated address channels, AR and AW respectively, which carry all the necessary address and control information for transaction.

The read data channel (R) is responsible for transferring both the read data and the corresponding response information from the slave to the master. It includes:

- A data bus, which can be 8, 16, 32, 64, 128, 256, 512, or 1024 bit wide
- A read response signal indicating the completion status of the read transaction

The write data channel (W) transfers ata from the master to the slave and includes:

- A data bus, with the same range of supported widths as the read data channel

- A byte lane strobe signal for every eight data bits, indicating which bytes in the ata word are valid

Data transmitted on the write data channel is treated as buffered, allowing the master to continue issuing write transactions without waiting for acknowledgements from the slave.

Finally, the write response channel (B) is used by the slave to indicate the completion of write transactions. Every write transaction requires a corresponding completion signal on this channel. As illustrated in Figures 2.1 and 2.2, the response is issued for the entire transaction, not for individual data transfers within the burst.

Throughout this work, each logical unit of communication on the AXI interface, including request, data and response, will be referred to as AXI transfer.

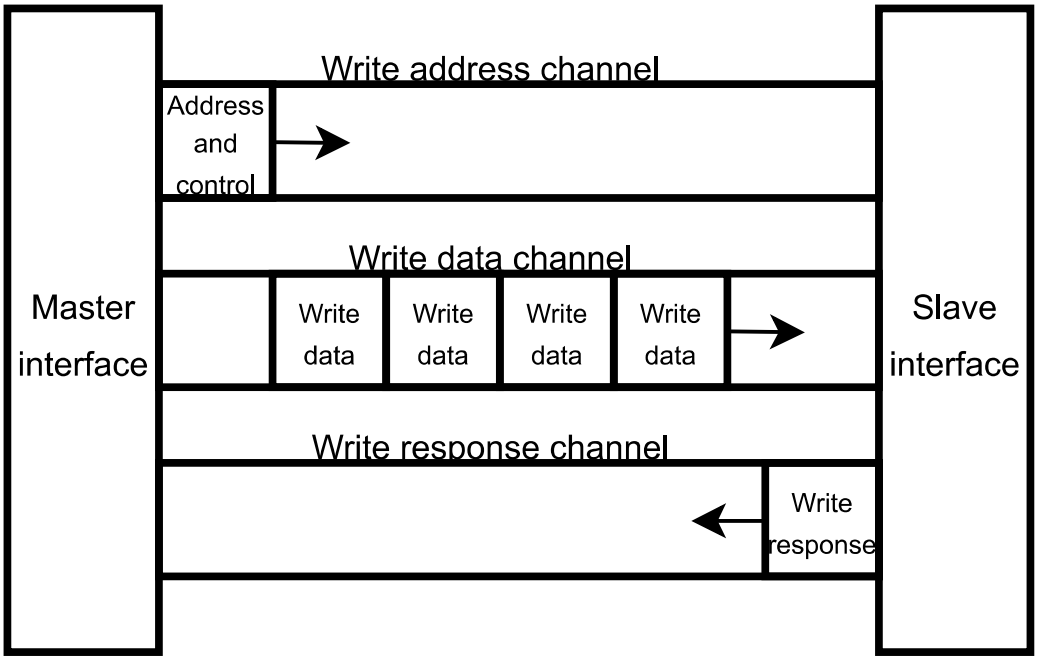


Figure 2.1: AXI channel architecture of writes. Taken from [1].

2.2 AXI-Stream

As detailed in [7], the AXI-Stream protocol is used as a standard interface for exchanging data between connected components. It is designed as a point-to-point protocol, connecting a single transmitter and a single receiver.

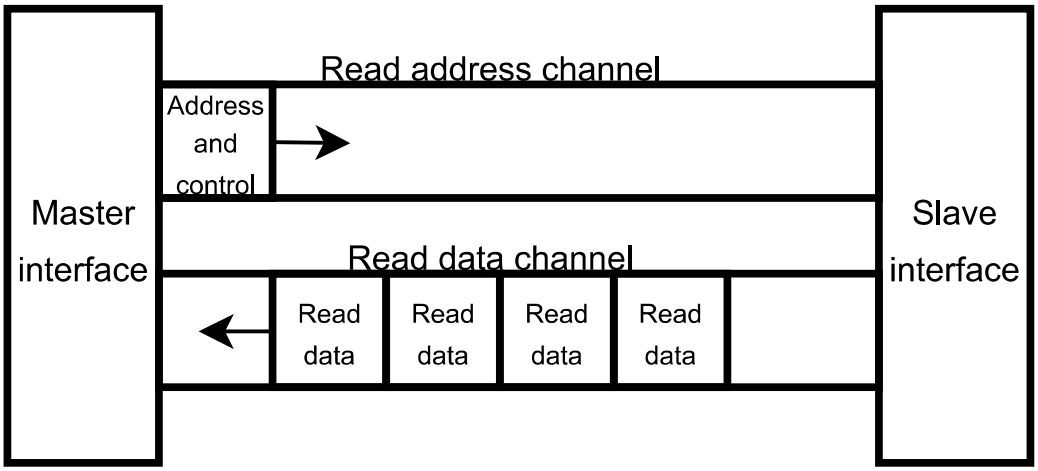


Figure 2.2: AXI channel architecture of reads. Taken from [1]

However, by using interconnects, the protocol is also applicable in systems with multiple master and slave components, allowing for scalable communication architectures. As discussed in [8], AXI-Stream is commonly used in high-bandwidth applications, serving as the interface for digital signal processing (DSP), communication, video and wireless IP cores.

In this work, AXI-Stream is employed as a high-bandwidth interconnect between the network layer and the data layer of the serial link.

Throughout this work, each logical unit of communication on the AXI-Stream interface, will be referred to as AXI-Stream transfer.

Chapter 3

Hardware Architecture

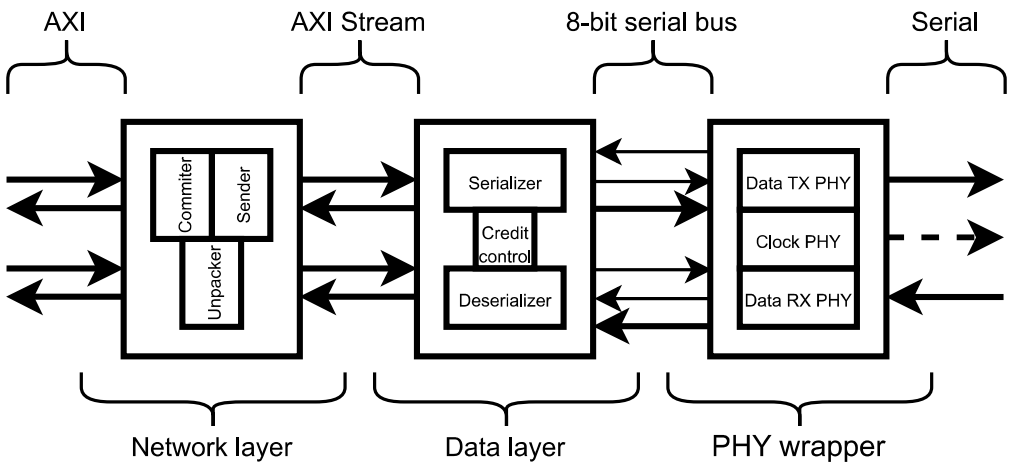


Figure 3.1: Overview of the AXI serial link. The network layer corresponds to OSI layer 3, the data layer to OSI layer 2 and the physical layer to OSI layer 1. Note how the translation from AXI to serial is divided into three translation stages.

As illustrated in Figure 3.1, the serial link is organized into three main stages, which correspond to OSI layers 1 through 3.

At the top of the stack, the system interfaces with AXI through the network layer. This layer is responsible for converting AXI transfers into AXI-Stream transfers. These transfers are then passed to the data layer, which serializes them onto an 8-bit wide bus.

In addition to serialization, the data layer also handles credit-based flow control, ensuring reliable data transfer under varying throughput conditions.

Once processed, the 8-bit wide stream is forwarded to the PHY, where it undergoes further serialization onto a single data wire suitable for serial transmission. This stage also transmits the data alongside the transmitter clock over the serial connection.

To optimize power usage, the PHY is encapsulated within idle control logic. This allows the system to reduce power consumption by setting the transmitter PHY into a low-power idle mode when the link is not transmitting.

Incoming serial data follows the reverse path, being deserialized and converted back into AXI transfers at the AXI interface.

3.1 Network layer

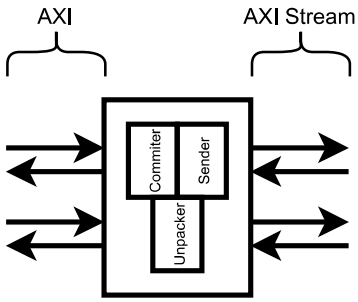


Figure 3.2: Overview of the network layer. The AXI to AXI-Stream pipeline is processed by the committer and sender, while the AXI-Stream to AXI pipeline is processed by the unpacker.

The network layer architecture closely follows the implementation from [6].

This layer is responsible for interfacing with both AXI and AXI-Stream protocols, handling their respective handshake mechanisms, and performing translation between the two formats.

In the AXI to AXI-Stream direction, the conversion process involves packing the AXI data together with the type of AXI channel into a single payload. This payload also includes the B channel, ensuring that write responses are always transmitted. The resulting payload is then sent as the AXI-Stream data, accompanied by any necessary sideband signals.

For the reverse direction, AXI-Stream transfers are unpacked and the extracted data is routed to the appropriate AXI channels.

These operations are distributed among three main components: the committer, the sender and the unpacker.

The committer collects incoming AXI transfers and prepares them for conversion. Once prepared, the sender translates the committed transfers into AXI-Stream format and forwards them to the data layer for serialization.

On the receiving end, the unpacker processes incoming AXI-Stream transfers, reconstructs the original AXI transfers, and outputs them from the link in AXI format.

3.2 Data layer

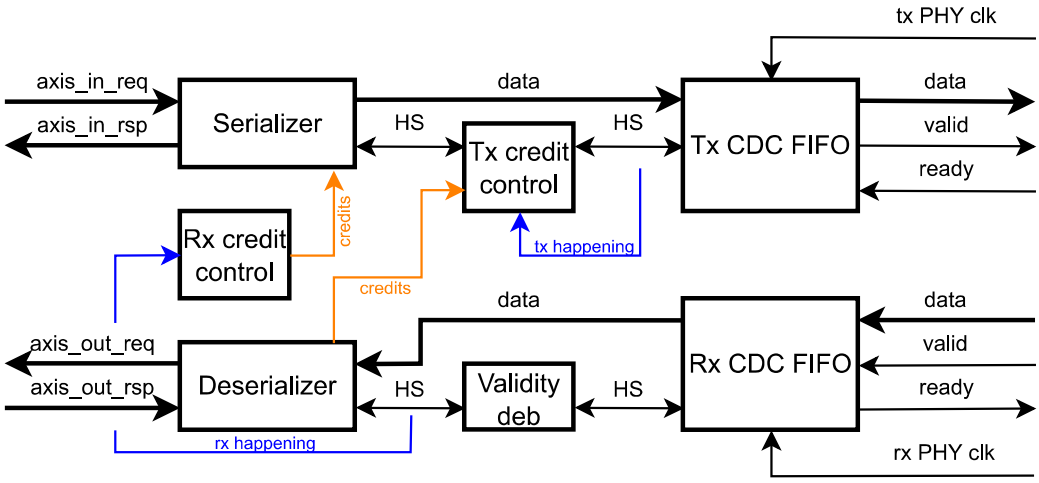


Figure 3.3: Overview of the data layer. The serializer/deserializer convert between AXI-Stream and the 8-bit serial bus. CDC FIFOs handle buffering and clock domain crossing. Credit-based flow control manages backpressure, while the validity debouncer mitigates transmission artefacts.

The data layer interfaces between the AXI-Stream protocol and the 8-bit serial bus connected to the PHY wrapper. It performs bidirectional conversion between AXI-Stream an the serial bus format.

On the transmitter side, this is achieved using a serializer, which converts the AXI-Stream data into the 8-bit format required for transmission. Conversely, on the receiver side, a deserializer reconstructs the incoming 8-bit serial stream back into AXI-Stream format.

At the beginning of any serial transmission, the receiver-side clock domain crossing (CDC) first-in first-out (FIFO) buffer incorrectly treats the first two bytes of data as valid,

even though they are invalid. This behavior arises from the manner in which the PHY transmits data, as described in Section 3.3.

To ensure correct handling of this transmission artefact, the first two bytes of data are explicitly marked as invalid by the validity debouncer. This mechanism is implemented as a finite state machine, illustrated in Figure 3.4.

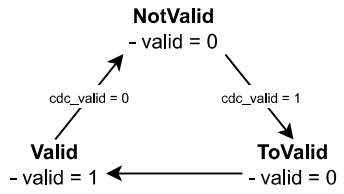


Figure 3.4: Finite state machine of the validity debouncer. The arrow without label indicates that the transition happens in any case.

Since the PHY wrapper operates on the PHY clock, it resides in a different clock domain than the rest of the serial link logic. As a result, CDC FIFO buffers are required to safely transfer data across the clock domain bounaries, between the serializer an the PHY wrapper on the transmitter side, an between the PHY wrapper and the deserializer on the receiver side.

To prevent overflow of the CDC FIFO on the receiver side, a credit-based flow control mechanism is employed. This is also implemented within the data layer. It operates by exerting backpressure on the transmitter: the handshake signals between the transmitter-side CDC FIFO an the serializer are interrupte whenever continuing transmission woul risk overflowing the receiver's CDC FIFO.

3.2.1 Credit-based flow control

The serial link employs two wires: one dedicated to forwarding the transmitter clock and the other used for transmitting the data stream. As a result, no conventional handshaking takes place between the transmitter and receiver.

To still enable backpressure and prevent overflow of the receiver-side CDC FIFO buffer, the data layer implements a credit-based flow control mechanism. The main signals involved in this scheme are illustrated in Figure 3.3.

Flow control begins with a predefined number of credits available at the transmitter. For each 8-bit packet inserted into the transmitter's CDC FIFO buffer on the master side, the available credit count is reduced by one. When no credits remain, the handshake between the serializer and the CDC FIFO buffer is broken, halting further data insertion.

The initial credit value is chosen to guarantee that transmission only occurs when there is sufficient storage space in the receiver CDC FIFO. Credits are replenished once data is removed from the receiver CDC FIFO buffer on the slave side.

Each time a data word is popped from the receiver CDC FIFO, the count of credits to be sent back to the master increases by one. Upon transmission and successful reception of these credits by the master, they are added back to the transmitter's credit pool, allowing further data to be sent.

3.2.2 Serializer

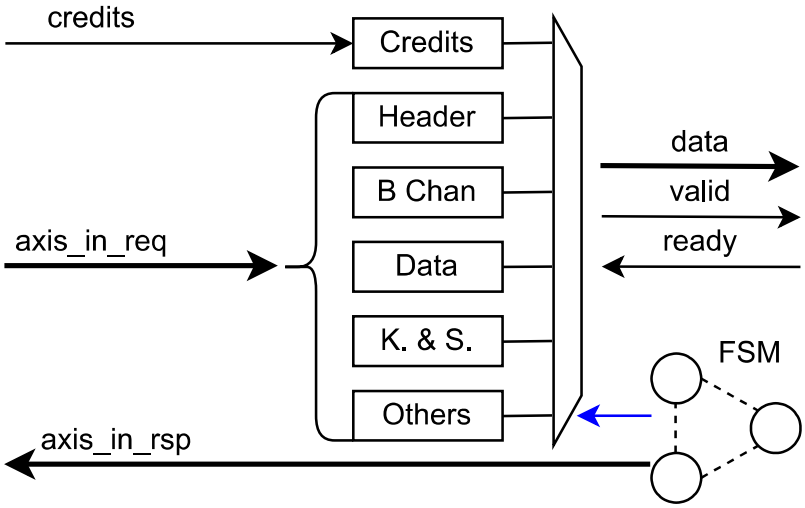


Figure 3.5: Overview of the serializer. The incoming AXI-Stream flits are divided into packets and stored into FFs. The FSM-controlled multiplexer then sequentially selects 8 bit packets to be transmitted, serializing the AXI-Stream flit. Note that the credits to transmit have a dedicated packet.

As shown in Figure 3.5, the serializer is built around a finite state machine (FSM)-controlled multiplexer that feeds 8 bit packets of AXI-Stream data into the 8 bit serial bus.

When a valid AXI-Stream flit arrives at the serializer, it is temporarily stored in flip-flops (FFs). The FSM then activates the multiplexer, which sequentially selects 8 bit partitions of the stored flit to be forwarded to the transmitter CDC FIFO buffer. Once all valid partitions have been transmitted, the serializer asserts the ready signal on the AXI-Stream interface, indicating that the input FFs can be overwritten with the next valid flit.

The FSM governing the serializer operates through several states as illustrated in Figure 3.6.

In the `Header` state, the serializer transmits the message type. This identifier informs the slave about the AXI channel associated with the data being transmitted. Additionally, it signals whether a valid B channel is included in the payload, whether the B channel is the only content, or whether the message is a credit-only transmission used to send available credits back to the master without any AXI-Stream payload.

The `Credits` state handles the transmission of credits from the slave to the master, replenishing the master's credit count. Once the credit transmission is complete, the FSM returns to the `Header` state.

If the AXI-Stream payload includes a valid B channel, it is transmitted in the `BChannel` state. If the payload contains only a B channel, the FSM completes the transmission and reverts to the `Header` state. If no valid B channel is present, the `BChannel` state is skipped entirely.

Payload data is handled in the `Data` state, where it is transmitted at a rate of 8 bit packets per clock cycle. Given that a payload may span multiple clock cycles, the FSM regularly transistions into the `IntervalCredits` state. This state transmits additional credits to minimize the duration in which the the transmitter is stalled due to lack of available credits.

Once all payload data has been sent, the FSM proceeds to the `KeepAndStrb` state and then the `Others` state. These final states transmit the keep, strobe, ID, destination, user and last signals of the AXI-Stream protocol. Although these sideband signals are unused in this design, the states are retained as placeholders to preserve AXI-Stream protocol compliance and ensure extensibility.

After completing this sequence, the FSM returns to the `Header` state and reasserts the ready signal on the AXI-Stream interface, signaling readiness for the next flit.

3.2.3 Deserializer

The deserializer serves as the receiver-side counterpart to the serializer, reassembling incoming 8 bit packets into complete AXI-Stream flits. As illustrated in Figure 3.7, the core of the deserializer is an FSM-controlled demultiplexer that routes valid 8 bit packets into FFs. The FSM sequence mirrors that of the serializer, ensuring that each packet received at a particular deserializer state was transmitted at the same state at the serializer. This allows for proper reconstruction of the original AXI-Stream flit.

The FSM begins operation in the `Header` state, as detailed in Figure 3.8. The first valid 8 bit packet received is the message type, which encodes the AXI channel of the upcoming transmission. As described in Section 3.2.2, the message type also conveys whether a

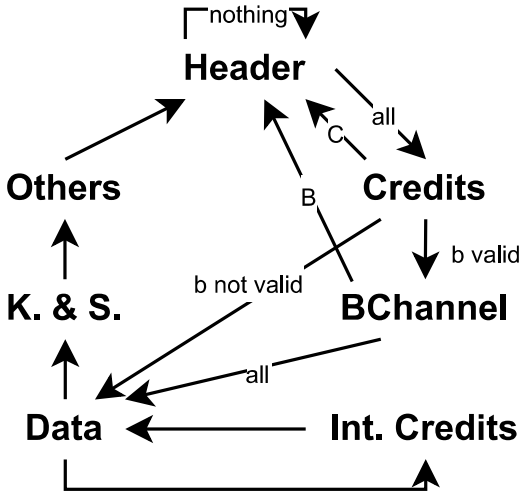


Figure 3.6: Finite state machine controlling the serializer. The labels on the arrows indicate the message type which triggers the transition. Arrows without label indicate that the transition can happen regardless of message type.

valid B channel is included, whether the payload consists solely of a B channel, or whether it is a credit-only transmission.

Following the `Header` state, the FSM enters the `Credits` state to process the second packet, which contains the free credits being returned to the master. These are added to the transmitter's credit count. If the message type indicates a credit-only transmission, the FSM immediately returns to the `Header` state to await the next message.

If the message type signals the presence of a valid B channel, the FSM proceeds to the `BChannel` state to receive it. In the case the transmission exclusively contains the B channel, the FSM returns to the `Header` state afterward. If not, it transitions to the `Data` state for further receiving. Alternatively, if the message type indicates that no B channel is present, the FSM skips the `BChannel` state and enters the `Data` state directly.

In the `Data` state, the FSM receives the primary data payload in multiple of 8 bit packets. To ensure a consistent flow of credits and prevent the transmitter from stalling, the FSM periodically enters the `IntermediateCredits` state. During this state, credits are received and added to the transmitter's available credits.

Once the data section is fully reconstructed, the FSM transitions to the `KeepAndStrobe` state, followed by the `Others` state. As explained in section 3.2.2, these final states transmit the keep, strobe, ID, destination, user and last signals of the AXI-Stream protocol, and serve as placeholders for future extensions.

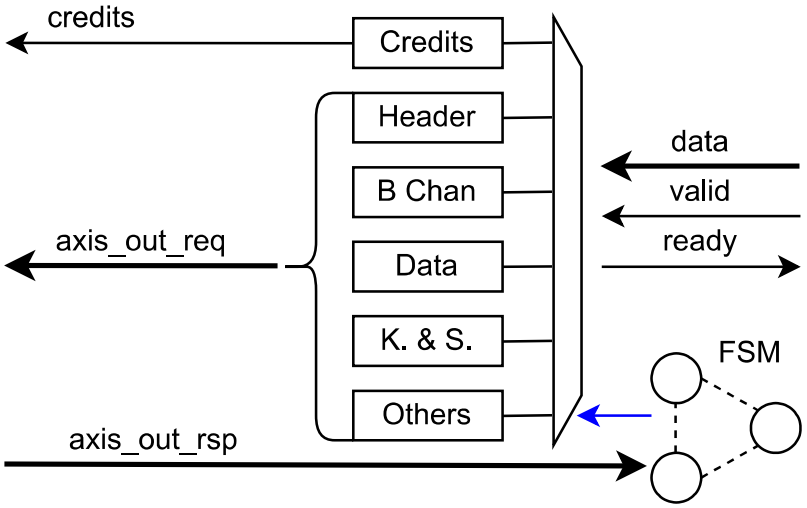


Figure 3.7: Overview of the deserializer. Incoming serial data is spread into packets by the FSM-controlled multiplexer, reconstructing the AXI-Stream flit and thus deserializing the data. As in the case of the serializer, the credits are handled separately from the rest of the AXI-Stream flit.

If the AXI-Stream interface is not ready to accept new data, the FSM transitions to the `Wait` state. This ensures that the reconstructed flit remains stored in the FFs and the `valid` signal is asserted until the interface signals readiness.

Once the AXI-Stream interface is ready, the FSM resets to the `Header` state, enabling the deserializer to begin reconstructing the next flit.

3.3 PHY wrapper

As shown in Figure 3.9, serial transmission in each direction uses two wires: one for data and one for forwarding the transmitter clock. Each of these wires connects to its own PHY instance.

The data wire carries the actual serial transmission, while the accompanying clock wire forwards the transmitter clock. On the receiver side, both wires are connected to a receiver PHY, which reconstructs the original signal using the forwarded clock. The transmitter PHY translates the 8-bit bus into a 1 bit data stream, while the receiver PHY converts the 1 bit stream back into a 8-bit wide serial bus.

Both the transmitter and receiver PHYs require a warmup period after a reset before they can be used for data transmission. This is ue to the internal clock generation within the PHYs, which must be initialized before they are operational. The warmup procedure

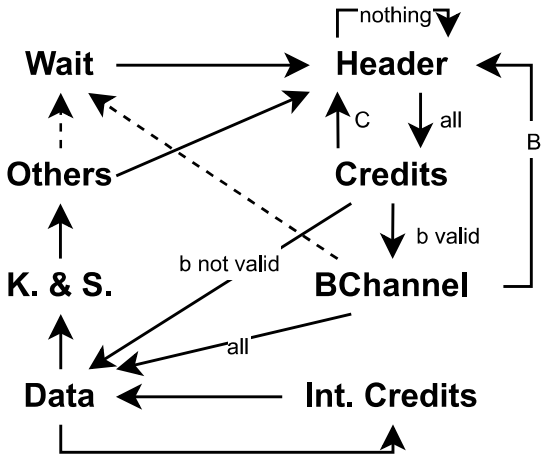


Figure 3.8: Finite state machine controlling the deserializer. The labels on the arrows indicate the message type which triggers the transition. Arrows without label indicate that the transition can happen regardless of message type.

is controlled by dedicated FSMs on both the transmitter and receiver sides, shown in Figures 3.10a and 3.10b.

The warmup process begins immediately after reset. The transmitter FSM starts in the `Off` state, uring which it transmits a preefined training sequence over the data line and forwards the clock. After a fixed number of clock cycles, the FSM transitions to the `Idle` state. At this point, the PHY is assumed to be fully initialized, and training and clock forwarding are halted. The `Idle` state is also entered during periods of inactivity as a power saving feature.

On the receiver side, the FSM starts in the `RxOff` state and transitions to the `Run` state after a fixed number of clock cycles. Since no clock is forwarded during the transmitter's `Idle` state, the receiver CDC FIFO, which is clocked by the forwarded clock, cannot push any data. This absence of activity is used to distinguish between valid and invalid data, as the receiver FSM continues to assert the `valid` signal during `Idle`, even though no actual data is being received.

When valid data becomes available on the transmitter side, the FSM immediately transitions to the `Run` state. Crucially, the clock forwarding resumes one cycle before data transmission begins. This ensures that the first data byte is correctly latched by the receiver. Conversely, once data transmission ends, the FSM returns to the `Idle` state, but clock forwarding is only stopped one cycle after the last byte has been sent. This guarantees that the final byte is properly captured by the receiver's CDC FIFO.

Due to the early and late forwarding of the clock, two bytes of invalid data are interpreted

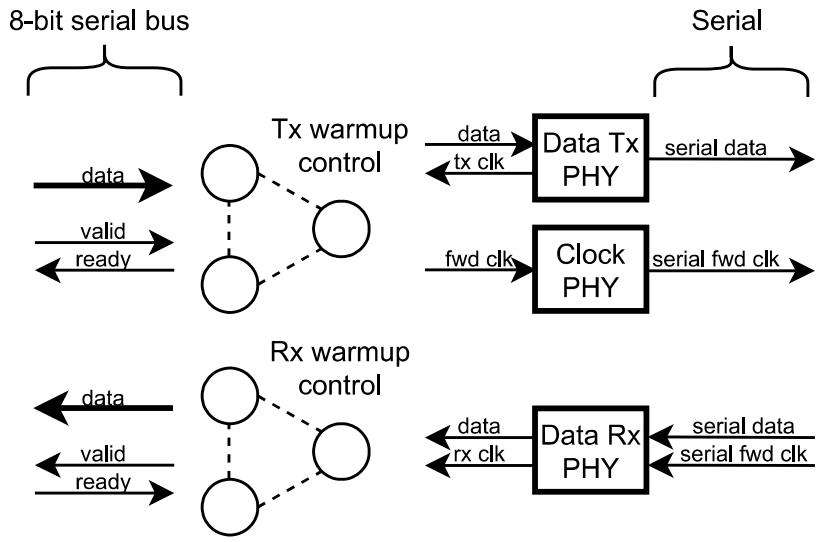
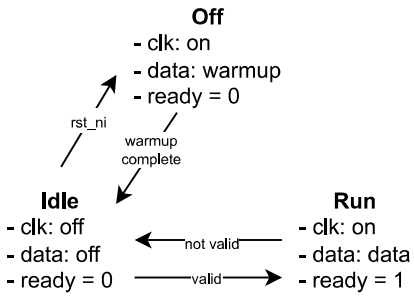
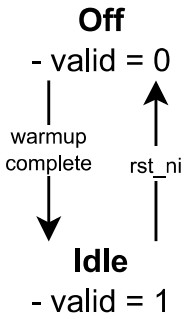


Figure 3.9: Overview of the PHY wrapper.

as valid and pushed into the receiver FIFO. This artefact motivates the use of the validity debouncer FSM described in Section 3.2.



(a) Finite state machine controlling the tx PHY warmup.



(b) Finite state machine controlling the rx PHY warmup.

Chapter 4

Results

4.1 Implementation results

The implementation was carried out using *SystemVerilog* code and compiled and simulated with *QuestaSim 2019.3*. The serial link testbench, including the AXI master driver and slave, is based on components from the previous serial link implementation [6].

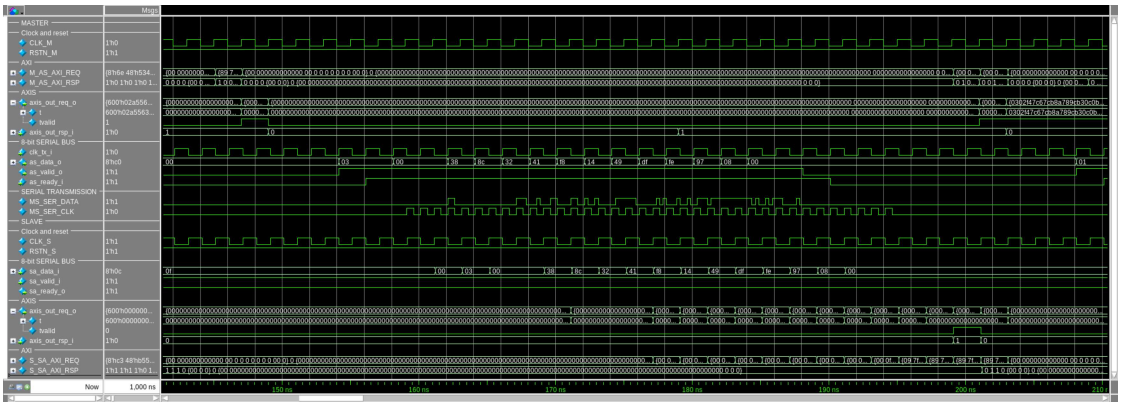


Figure 4.1: Master to slave transmission overview.

Figures 4.1, 4.2, and 4.4 are obtained by configuring the serial link such that the transmitter CDC FIFO depth is set to 8, and the receiver CDC FIFO depth is set to 64. The initial credit count for both master and slave is 72, matching the cumulative FIFO depth accross transmitter and receiver, ensuring that FIFO overflow cannot occur. Additionally, intermediate credits are configured to be transmitted every 12 data bytes.

Figure 4.1 illustrates a complete AXI transaction over the serial link, capturing each stage of the transmission in detail. The sequence begins at 145 ns, where a valid AXI transac-

tion is initiated, as indicated by the assertion of the `M_AS_AXI_REQ` and `M_AS_AXI_RSP` signals. At 149 ns, the network layer converts the AXI transfer into AXI-Stream flits, observable on the `axis_out_req_o` and `axis_out_rsp_o` signals.

This flit is then handed off to the data layer, where serialization takes place. Starting at 154 ns, the data begins to appear on the 8-bit serial bus, signaled by the activity on `as_data_o`, accompanied by the appropriate handshake signals `as_valid_o` and `as_ready_i`. The serialized data is then transmitted over the physical serial link, which becomes visible at 159 ns on the `MS_SER_DATA` line, alongside the forwarded clock `MS_SER_CLK`.

By 161 ns, the data reaches the receiving end, entering through the receiver's 8-bit serial interface via the `sa_data_i` signal and corresponding handshaking given by `sa_valid_i` and `sa_ready_o`. Over the next several cycles, the receiver deserializes the data and reconstructs the original AXI-Stream flit. This process is completed by 201 ns, once again visible on the `axis_out_req_o` and `axis_out_rsp_i` signals. Finally, at 203 ns, the AXI transfer concludes on the slave side with the activation of the `S_SA_AXI_REQ` and `S_SA_AXI_RSP`, marking the end of the transmission sequence.

Note that forwarded clock given by signal `MS_SER_CLK` is only active during actual data transmission. This behavior is implemented as a power-saving feature, reducing unncessary clock activity on the serial line.

4.1.1 Data layer implementation results

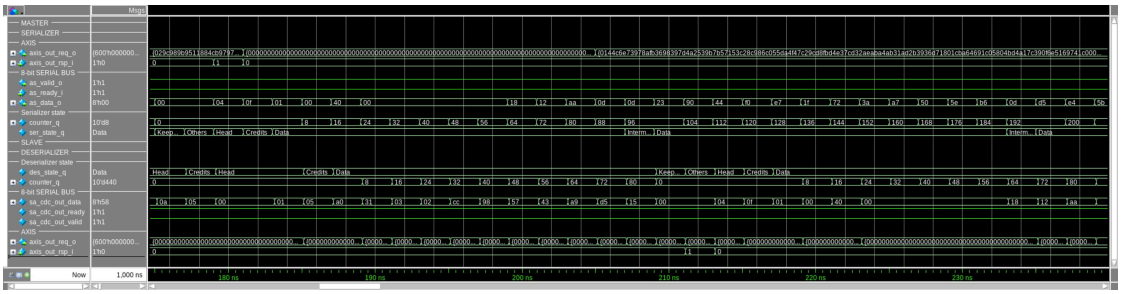


Figure 4.2: Master to slave transmission, detailing internal signals of the serializer and deserializer.

Figure 4.2 provides an in-depth look at the serialization and deserialization process taking place in the data layers of both the master and the slave. The transmission initiates at 181 ns with the serializer in the `Header` state as defined in the serializer FSM shown in Figure 3.6. The current state of the serializer is captured by the signal `ser_state_q`.

Immediately after, the serializer transitions into the `Credits` state and then into the `Data` state. This direct transition, bypassing any additional stages, is consistent with

the behavior outlined in Section 3.2.2, which specifies that such a flow occurs when the AXI-Stream payload does not contain a valid B channel.

By 207 ns, after twelve data bytes have been transmitted, the serializer enters the `IntermediateCredits` state to insert credits into the stream. Throughout this process, a counter given by signal `counter_q` at the transmitter side tracks the number of bytes sent during the `Data` state. An analogous counter at the receiver performs the equivalent function during reception. Notably, this counter is intentionally not reset during the `IntermediateCredits` state to maintain an accurate count of the data associated with a given AXI transfer.

On the receiving end, the deserializer completes processing of a preceding transmission and sequentially passes through the `KeepAndStrobe` and `Others` states before reaching the `Head` state at 213 ns. From that point onward, it begins registering the new data sent from the transmitter starting at 181 ns. It is worth emphasizing that the sequence of state transitions is identical in both the serializer and deserializer. This symmetry is fundamental to the integrity of the transmission, ensuring correct alignment and interpretation of data on both ends.

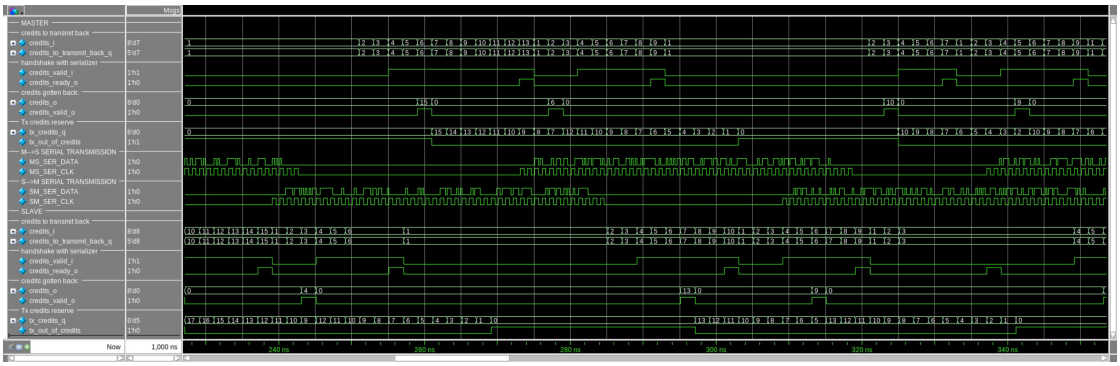


Figure 4.3: Credit based flow control in action.

Figures 4.3 and 4.5 are obtained by configuring the serial link such that the transmitter CDC FIFO depth is set to 8, while the receiver CDC FIFO depth is 16. To make the dynamics of flow control more visible, the initial number of credits on both the master and slave sides is set to 20. This is deliberately below the cumulative FIFO capacity, to trigger credit depletion events without risking FIFO overflow. Intermediate credits are configured to be sent after every 9 transmitted data bytes.

As shown in Figure 4.3, the top part of the waveform highlights signals related to the master's credit control, while the bottom half focuses on the slave. Transmission activity across the serial link is captured in the center of the figure.

The master begins transmitting to the slave at 230 ns, as evidenced by activity on the `MS_SER_DATA` and `MS_SER_CLK` signals. As the slave receives and processes this data, it consumes entries from its receiver CDC FIFO, which in turn increases the number of credits available to send back to the master. This increment is reflected in the

`credits_to_transmit_back_q` signal. At 237 ns, once the threshold is reached, the slave indicates readiness to send credits via the `credits_ready_o` signal. The corresponding value is passed to the serializer through `credits_i`, and the credit count is decremented.

The current number of available credits for transmission at each endpoint is indicated by `tx_credits_q`. For example, at 230 ns, this value is 16 at the slave side, allowing for 16 more bytes to be transmitted. Starting from 237 ns, the slave begins transmitting to the master. As this continues, `tx_credits_q` decreases until 243 ns, at which point additional credits from the master arrive via the `credits_o` signal.

Later, at 251 ns, the master starts consuming data from its own receiver CDC FIFO, causing its `credits_to_transmit_back_q` to incease accordingly. The credits transmitted by the slave earlier, at 237 ns, arrive at the master at 260 ns and are added to the master's `tx_credits_q` reserve.

By 269 ns, the slave reaches a point where its transmitter runs out of credits. This is reflected by the assertion of `tx_out_of_credit`, a signal that prevents further data from being pushed into the slave's transmitter CDC FIFO. This mechanism ensures that the master's receiver CDC FIFO does not overflow. The resulting backpressure causes the slave to stop transmitting over the serial link at 284 ns. Transmission resumes at 309 ns, once additional credits are received and `tx_out_of_credits` is deasserted.

4.1.2 PHY wrapper implementation results

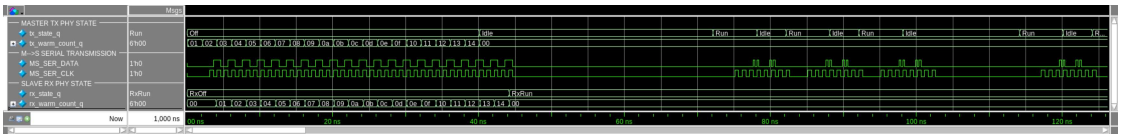


Figure 4.4: PHY initial warmup phase.

Figure 4.4 illustrates the behavior of the PHY wrapper FSMs, as introduced in Section 3.3, with particular emphasis on the initial warmup phase. Immediately following reset, both the transmitter and receiver PHYs are not available for data transmission. This is indicated by the transmitter's state signal `tx_state_q` being in the `Off` state and the receiver's `rx_state_q` being in the `RxOff` state.

During this initial phase, the transmitter initiates a warmup sequence by transmitting an arbitrary training pattern over the `MS_SER_DATA` signal, synchronized with the forwarded clock on `MS_SER_CLK`. While the warmup phase does not serve a functional role in the present simulation, it acts as a placeholer for future implementations of the low-power serial link where such behavior might be required.

During warmup, both the receiver and transmitter maintain internal warmup counters, `rx_warm_count_q` and `tx_warm_count_q` respectively, which increment in response to the

forwarded training sequence until a predetermined count of 20 clock cycles is reached. Once the warmup period concludes, the transmitter transitions to the `Idle` state at 40 ns, while the receiver enters the `RxRun` state at 44 ns, indicating readiness for standard data transmission.

4.1.3 Known issue

As outlined in Section 3.3, the PHY design requires that the transmitter forward its clock over the serial link one clock cycle earlier and one clock cycle later than the data payload. While this strategy is necessary for correct data capture at the receiver, it causes the receiver CDC FIFO to interpret and push two invalid bytes as valid at the start of each transmission.

Section 3.2 describes the use of the validity debouncer within the data layer that addresses this artefact by marking the first two bytes from the receiver CDC FIFO output as invalid. This correction mechanism only functions if the handshake signal between the CDC FIFO and deserializer output given by `sa_cdc_direct_valid` transitions from low to high, as would occur when the FIFO has no valid data to present at the start of transmission.

Now consider the following problematic scenario:

- Suppose a transmission over the serial link is interrupted for an arbitrary number of clock cycles and subsequently resumes, causing the CDC FIFO to again misinterpret two invalid bytes as valid once transmission resumes.
- Further assume that the receiver CDC FIFO is not empty at this time and continues presenting valid data. Consequently, the handshake signal `sa_cdc_direct_valid` remains asserted throughout the transmission interruption.
- Under these conditions, the validity debouncer fails to detect the start of a new transmission and thus does not flag the invalid bytes. As a result, these bytes are erroneously propagated further into the receiver pipeline, causing transmission errors.

This issue is shown in Figure 4.5. At 960 ns, the master briefly interrupts the transmission, which resumes at 964 ns. This interruption is visible in the signals `as_data_o` and `as_valid_o`, and is also reflected in a gap on the serial link, seen in `MS_SER_DATA` and `MS_SER_CLK`. The interruption results in two invalid bytes being inserted into the receiver CDC FIFO at 967 ns, as shown by `sa_cdc_out_valid`.

At 973 ns, the receiver CDC FIFO is unable to output valid data and deasserts `sa_cdc_direct_valid`, allowing the debouncer to identify the invalid region. When `sa_cdc_direct_valid` reasserts at 975 ns, the validity debouncer correctly handles this by marking the output as invalid until 979 ns, as shown by `sa_cdc_out_valid`.

Now consider a second scenario at 990 ns, where the master again momentarily halts transmission. This short interruption causes two invalid bytes to enter the receiver CDC FIFO at 999 ns and 1000 ns. However, in this case, the interruption is so brief that the transmitter PHY does not enter the `Idle` state, and the serial clock signal `MS_SER_CLK` remains uninterrupted.

Since the receiver CDC FIFO continues to output valid data, the `sa_cdc_direct_valid` signal does not deassert, and the validity debouncer is unaware of the new transmission boundary. As a result, `sa_cdc_out_valid` remains high, and the two invalid bytes are incorrectly propagated into the downstream logic, ultimately resulting in data corruption.

Although this issue is solvable, it persists in the current system architecture due to its discovery late in the design phase.

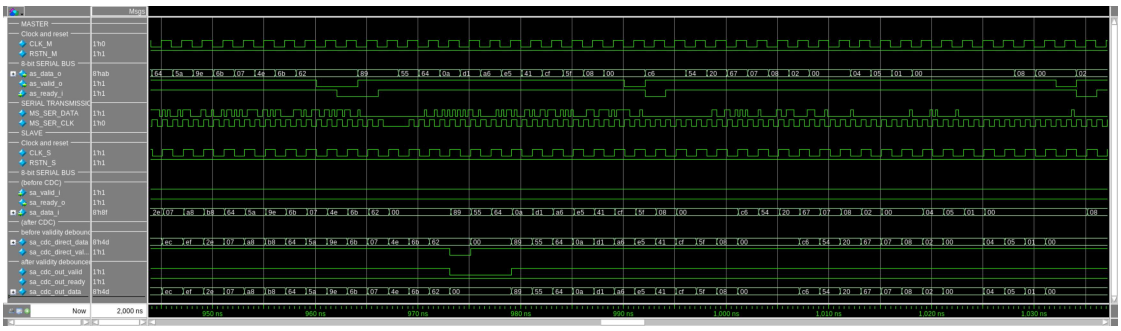


Figure 4.5: Master to slave transmission shows the known problem with the current architecture.

4.2 Performance figures

The architecture presented in this work has not yet entered the synthesis phase. Therefore, precise area and power figures are not available at this time. However, as the PHY's power consumption is already known, an estimate of the serial link's overall power usage can be made regardless. Similar to the previous implementation described in [6], the PHY dominates power consumption, allowing the rest of the serial link's power usage to be neglected in preliminary estimates.

The serial link operates at a frequency of 500 MHz. Given that the PHY processes 8 bits per clock cycle, this results in a maximum theoretical throughput of 4 Gbps. The PHY's energy consumption is 500 fJ/b, which is approximately 3.2× lower than the previous serial link design, which consumed 1.6 pJ/b [6]. Consequently, operating at the maximum throughput of 4 Gbps, the PHY would consume roughly 2 mW.

The utilization of the serial link epends on both the frequency of intermediate credit transmissions and the channels involved in the current AXI transfer. Nonetheless, a reresentative estimate can be provided.

Referring to the serializer state machine in Figure 3.6, every AXI transfer involves transmitting an 8-bit message type packet during the `Header` state an a 8 bit credit packet during the `Credits`. Assume intermediate credits are sent every 12 clock cycles and that the AXI data width is 512 bit. This configuration results in approximately $512/12 \approx 43$ intermediate credit transmissions. Additionally, assume no B channel data is inclued in the AXI-Stream payload.

Under these assumptions, the effective link utilization can be approximated by:

$$\text{Utilization} = \frac{512}{8 + 8 + 43 \cdot 8 + 512} \approx 58.7\,\%.$$

This utilization is lower than the 96% achieved in the previous serial link implementation [6]. The reduction is hypothesized to stem from a less efficient mapping of the AXI-Stream payload into 8 bit packets by the current serializer architecture. Notably, each credit packet occupies a full 8 bit slot, enabling the transmission of up to 255 credits in a single packet. This is far more than typically required, and even exceeds the total number of credits present in the system. As a result, the first few most significant credit packet bits are always unused, thereby reducing overall link efficiency in this implementation.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This work presents the implementation and simulation of a low-power, AXI4-interfaced serial link, based on the architecture proposed in [6]. Key contributions include a refined network layer, the development of a new data layer, and a new PHY wrapper.

The modified architecture facilitates the integration of a low-power mixed-signal PHY, which achieves high throughput while consuming significantly less energy than the all-digital PHY presented in [6]. The resulting serial link operates at a maximum theoretical throughput of 4 Gbps while consuming only 2 mW, corresponding to an energy-per-bit of 500fJ/b. This represents an approximate 3.2× improvement over the previous design.

Power efficiency is further enhanced by the inclusion of an idle mode in the PHY wrapper. This mode disables transmission in the absence of valid data, thereby minimizing dynamic power consumption during inactivity. Moreover, the serial link implements backpressure through credit-based flow control without requiring explicit handshakes over the link.

Simulation results validate the correct operation of the architecture under ideal conditions, and illustrate the impact of known limitations such as those caused by invalid byte registration due to short transmission interruptions. Despite these challenges, the architecture serves as a solid foundation for future low-power, high-thoughput AXI-interfaced serial links.

5.2 Future work

While the current implementation achieves promising performance and energy efficiency, several avenues for improvement and extension remain:

- **Clock-Data Phase Calibration:** Although the architecture includes a warmup sequence post-reset, it lacks a calibration mechanism to compensate for clock-data phase misalignment. To ensure robustness under realistic conditions, the architecture should incorporate an adaptive calibration procedure. This would involve the generation and detection of tranining sequences as well as tunable bit-shifting in the receiver path to recover properly aligned data.
- **Serializer Efficiency:** The current serializer partitions AXI-Stream data into fixed 8 bit segments. This fixed-width strategy introduces inefficiencies, particularly for credit transmissions, which rarely require the full 8 bits. Implementing an improved and more flexible encoding scheme could significantly improve link utilization.
- **Correction of Known Receiver Bug:** As discussed in Section 3.3 and illustrated in Figure 4.5, the receiver CDC FIFO may misinterpret invalid data as valid under short transmission interruptions. A more robust synchronization mechanism between master serializer and slave deserializer is required to detect an discard invalid data regardless of CDC FIFO output status.
- **Scalability via Multi-Channel PHYs:** To support higher throughput applications, the architecture can be extended by instanciating multiple parallel PHY channels. A channel allocator and reordering logic would be necessary to preserve packet order and distribute the load efficiently accross the links. Such scaling would enable multi-Gbps aggregate throughput while leveraging the same low-energy PHY design.
- **Synthesis and Physical Design Exploration:** As the current design is pre-synthesis, future work should include the RTL- to-GDSII flow implementation. This would provide accurate area, timing and power figures, as well as insights into physical constraints.

Overall, the proposed architecture demonstrates the viability of low-power-high-speed serial-links using the novel PHY. With appropriate extensions, it holds promise for integration into broader systems-on-chip or chiplet-based platforms.

Bibliography

[1] A. Ltd. (2013) Amba axi and ace protocol specification. Version E. [Online]. Available: https://developer.arm.com/documentation/ihi0022/e/

[2] P. Pannuto, Y. Lee, Y.-S. Kuo, Z. Foo, B. Kempke, G. Kim, R. G. Dreslinski, D. Blaauw, and P. Dutta, "Mbus: A system integration bus for the modular microscale computing class," *IEEE Micro*, vol. 36, no. 3, pp. 60–70, 2016.

[3] Z. Meng, J. Deng, M. Zhang, S. Song, Z. Liu, and J. Feng, "Design and implementation of ultra-low power i2c-compatible bus based on asynchronous circuit," in *2023 6th International Conference on Electronics Technology (ICET)*, 2023, pp. 79–84.

[4] A. Gupta, S. Soni, S. K. Muthukumar, D. J. Kurian, and V. Parvatha, "Low power, light weight and transparent multi-chip communication link for iot segment," in *2021 IEEE 18th India Council International Conference (INDICON)*, 2021, pp. 1–6.

[5] H. Okuhara, A. Elnaqib, M. Dazzi, P. Palestri, S. Benatti, L. Benini, and D. Rossi, "A fully integrated 5-mw, 0.8-gbps energy-efficient chip-to-chip data link for ultralow-power iot end-nodes in 65-nm cmos," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 10, pp. 1800–1811, 2021.

[6] P. Scheffler, T. Benz, V. Potocnik, T. Fischer, L. Colagrande, N. Wistoff, Y. Zhang, L. Bertaccini, G. Ottavi, M. Eggimann, M. Cavalcante, G. Paulin, F. K. Gürkaynak, D. Rossi, and L. Benini, "Occamy: A 432-core dual-chiplet dual-hbm2e 768-dp-gflop/s risc-v system for 8-to-64-bit dense and sparse computing in 12-nm finfet," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 4, pp. 1324–1338, 2025.

[7] A. Ltd. (2021) Amba axi-stream protocol specification. ARM IHI 0051B. [Online]. Available: https://developer.arm.com/documentation/ihi0051/b/

[8] X. Inc. (2012, Nov) Axi reference guide. UG761 (v.14.3). [Online]. Available: https://docs.amd.com/v/u/en-US/ug761_axi_reference_guide